BioSpecInfo

Verification & Validation (V&V) & Benchmark Report — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v238** · commit `ecc7a88`

Generato il 2026-11-04

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Verification & Validation (V&V) & Benchmark Report — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Scopo	Documentare metodo di verifica, dati di validazione dei predittori spettrali, prestazioni cross-device e limiti dichiarati del modello.

1. Scopo e strategia di V&V

- **Verifica (Verification):** il software è costruito correttamente (sintassi, assenza di errori runtime, comportamento atteso dei componenti).
- **Validazione (Validation):** i risultati scientifici (predittori spettrali, descrittori) sono confrontati con banche dati sperimentali di riferimento.

2. Verifica del software (QA)

Tecnica	Descrizione	Stato
Controllo sintattico	`node --check` sui blocchi JavaScript critici a ogni modifica.	✅ In uso
Test end-to-end (E2E)	Automazione con **Playwright** su Chromium headless: apertura sezioni, disegno/caricamento molecole, verifica dei pannelli (notazioni, spettri, gruppi funzionali, ADMET, similarità), animazioni 2D/3D. Conteggio degli errori JavaScript di console/pagina (criterio di accettazione: **0 errori** non di rete).	✅ In uso
Verifica su server HTTP	RDKit WASM richiede contesto HTTP (CORS): i test girano su server locale per riprodurre l'ambiente reale.	✅ In uso
Test manuale cross-device	Prova su dispositivi reali (vedi §4).	✅ In uso
Suite di unit test automatizzata (Vitest/Jest)	—	🔜 Roadmap
Copertura ≥ 90%	—	🔜 Roadmap

> Ogni release è verificata E2E con Playwright con criterio "zero errori JS" prima del merge sul ramo principale.

3. Validazione scientifica dei predittori spettrali

I predittori spettrali di BioSpecInfo usano **modelli euristici ad additività di gruppo / regole strutturali** (tabelle di riferimento di letteratura), scelti per garantire **risposta istantanea in-browser** su qualsiasi dispositivo.

3.1 Benchmark ^{1}H-NMR vs SDBS

Metrica	Valore riportato
Database di riferimento	**SDBS** (Spectral Database for Organic Compounds, AIST)
Errore Assoluto Medio (MAE)	**0,31 ppm**
Segnali entro ±0,5 ppm	**89%**

3.2 Benchmark IR (banda C=O) vs NIST

Metrica	Valore riportato
Database di riferimento	**NIST Chemistry WebBook**
Errore Assoluto Medio (MAE)	**11 cm^{-1}**
Bande entro ±30 cm^{-1}	**90%**

3.3 Proprietà fisico-chimiche

Descrittori (peso molecolare, formula, LogP, TPSA, HBD/HBA, anelli, stereocentri) calcolati tramite **RDKit MinimalLib** e confrontabili con **PubChem** e **DrugBank**. Esempio di verifica riproducibile: *colesterolo → $C_{27}H_{46}O$, 4 anelli, 8 stereocentri*.

> **Fonte dei dati di benchmark:** misurazioni condotte dall'autore nell'ambito della tesi di laurea. I valori vanno intesi come validazione a scopo **didattico e di identificazione qualitativa** dei gruppi funzionali, non come sostituto di calcoli quantomeccanici di riferimento.

---

4. Prestazioni cross-device

Prove di fluidità dei rendering interattivi (viewer 3D, animazioni, editor) condotte su un ventaglio di dispositivi:

Dispositivo	Esito
iPhone / iPad	30–60 fps
Laptop Windows / macOS	60 fps
Raspberry Pi 4	30–60 fps

Peso del file principale `index.html`: **≈ 4,5 MB** (l'intero pacchetto, comprensivo dei moduli WASM e degli asset 3D, è superiore ma resta interamente statico e cache-abile offline).

5. Limiti dichiarati del modello

In coerenza con il principio di trasparenza scientifica:

1. **Modello ad additività.** I predittori NMR/IR sommano contributi di gruppo e

non modellano pienamente gli effetti dell'intorno chimico esteso (coniugazione lunga, effetti anisotropici, legami a idrogeno intramolecolari). Scarti maggiori sono attesi, ad es., su **carbonili fortemente coniugati** o su casi limite come l'**acido formico**.

1. **Spettri qualitativi/didattici.** Gli spettri simulati servono a riconoscere

i gruppi funzionali e a comprendere le relazioni struttura–spettro, non a sostituire dati sperimentali certificati.

1. **Dipendenza da rete per alcune funzioni.** Nome IUPAC/comune, numero CAS,

pittogrammi GHS e modelli 3D reali provengono da PubChem: in assenza di rete l'app mostra un fallback e resta pienamente funzionale sui dati locali.

1. **Precisione a doppia via.** Per l'uso rigoroso si rimanda ai link diretti

verso i database ufficiali (NIST, SDBS, MassBank, nmrshiftdb2) integrati nell'interfaccia.

---

6. Criteri di accettazione di una release

- ✅ `node --check` supera senza errori sui moduli modificati.
- ✅ Test E2E Playwright: 0 errori JS non di rete.
- ✅ Nessuna regressione visibile nelle sezioni toccate.
- ✅ Versione della cache del Service Worker incrementata.
- ✅ `tools/verifica-farmaci.js`: nessun difetto; ogni deviazione registrata con il motivo.
- ✅ `verifica_guida`: ciò che la documentazione promette esiste nel codice.

---

7. Batteria di verifica — stato alla versione `bsi-v238`

La verifica non è più solo end-to-end funzionale: comprende banchi dedicati alle proprietà che un test funzionale non osserva (stabilità nel tempo, degrado di rete, accuratezza dei dati).

7.1 Composizione

Famiglia	Banchi	Oggetto
Dati scientifici	`verifica-farmaci`, `test_spettri`, `test_assi`, `test_assi_canvas`, `test_costanti`, `audit_dati`, `test_v237`, `test_3d238`	struttura vs peso molecolare, riconoscimento gruppi funzionali, convenzioni degli assi, costanti fisiche; le venti sezioni di astrofisica, fisica e matematica contro valori noti o un secondo metodo; le schede 3D di 29 sezioni (311 controlli dei calcoli, ogni scheda aperta a 1280 e 390 px)

Famiglia	Banchi	Oggetto
Agente AI	`test_ko`, `test_404`, `test_503`, `test_firma`, `test_attesa`, `test_attesalunga`, `test_tetto`, `test_nucleo`, `browser_ko`, `browser_prova`	modello ritirato, CORS, sovraccarico, firme di ragionamento, tetto di token, attese prolungate, memoria dei fornitori irraggiungibili
Stabilità	`audit_stabilita`, `audit_promesse`, `audit_quota`, `test_filemanager`, `test_doppioinvio`, `test_sw`, `test_visore3d`	sessioni lunghe, promesse rifiutate, memoria esaurita, archivio non disponibile, invii sovrapposti, rete degradata, contesti WebGL
Interfaccia	`browser_reset`, `browser_proxy`, `browser_proxyui`, `browser_rdkit`, `browser_lab`, `browser_frontiera`, `test_aggiorna`, `test_controllaagg`, `test_spectra238`, `test_guidaproxy`	pannelli, proxy, laboratorio RDKit, aggiornamenti (anche il pulsante «Controlla» premuto nei tre casi: già aggiornata, versione nuova, sito irraggiungibile); console di Spectra Live su telefono e schermo grande, navigatore, livello e concentrazione, riconoscimento del creatore, occhi e sonno del volto
Coerenza	`verifica_guida`	corrispondenza documentazione ⟷ codice, marcatori di conflitto su 58 file

7.2 Proprietà verificate che un test funzionale non osserva

Proprietà	Metodo	Risultato misurato
Assenza di perdite di memoria	194 sezioni aperte per 5 giri, conteggio nodi DOM per giro	costruzione +26 876 nodi al primo giro, **+0 nei quattro successivi**
Sopravvivenza a memoria esaurita	`localStorage.setItem` forzato a lanciare	10 pagine su 10 restano operative
Degrado di rete (≠ assenza di rete)	richieste sospese 20 s, intercettazione a livello di contesto	risposta dalla cache in **3 507 ms** (soglia 3 500 ms)
Riproducibilità degli spettri	doppio disegno, confronto byte a byte	identici
Contesti WebGL	conteggio costruzioni del visore 3D su 10 molecole	da 7 a **1**

7.3 Note di metodo

Un banco di prova che non misura nulla **passa**, ed è il modo più insidioso di ottenere una falsa garanzia. Durante lo sviluppo della batteria sono stati individuati e corretti diversi casi di misura vacua, fra cui:

- `page.route` di Playwright **non** intercetta le richieste originate dal

Service Worker: serve `context.route`. Con la prima, la prova «rete degradata» misurava una rete perfettamente funzionante e concludeva in 16 ms.

- Sondare l'esaurimento di `localStorage` con una scrittura di un byte non prova

nulla: dopo la saturazione un byte trova sempre posto.

- Le tacche di un asse su `<canvas>` non sono leggibili come testo: vanno

raccolte intercettando `fillText`, e la riga più in basso è il *titolo* dell'asse, non le tacche.

Da qui la regola adottata: **ogni banco che simula una condizione deve contare quante volte la simulazione è realmente scattata**, e fallire se il conteggio è zero.